D

Dumbhanengue ERP

10 — Guide du développeur

Guide pratique pour exécuter, tester et faire évoluer Dumbhanengue ERP.

Version 1.1.1+3 · 16 août 2026

Dumbhanengue ERP

Table des matières

Prérequis 3

Exécuter le projet 3

- Pointer vers un autre projet Supabase (ex. : production) 3
- Définir le super admin 3
- Persistance active (Supabase × local) 4
- Signature de la release Android 4

Carte mentale du flux de développement 4

Conventions du projet 5

- Architecture (non négociable) 5
- Persistance 5
- SQLite pour de nouvelles données 5
- État (Riverpod) 5
- UI 6
- Tests 6

Ajouter un nouveau module (étape par étape) 6

Commandes utiles 6

Documents connexes 7

Prérequis

- Flutter 3.x (Dart SDK ^3.12.2) ;
- Chrome (pour exécuter sur le web) ;
- Windows/mobile/desktop pour la persistance SQLite ; le web utilise la mémoire (repli automatique).

Exécuter le projet

```
# Installer les dépendances
flutter pub get
# Générer le code (Freezed + json_serializable) — requis après avoir créé/modifié
# des classes annotées avec @freezed
dart run build_runner build --delete-conflicting-outputs
# Exécuter sur un appareil/émulateur (flavor dev + defines d'env)
flutter run --flavor dev --dart-define-from-file=env/dev.env
# Exécuter sur le web (repli en mémoire)
flutter run -d chrome --flavor dev --dart-define-from-file=env/dev.env
# Tests
flutter test
# Analyse statique
flutter analyze
```

Pointer vers un autre projet Supabase (ex. : production)

Les identifiants sont fournis via flavors + `--dart-define-from-file` (FLUTTER_APP_FLAVOR est réservé — injecté depuis --flavor) :

```
flutter build appbundle --flavor prod --release \
  --dart-define-from-file=env/prod.env
```

- env/dev.env est commité (projet dev) ; env/prod.env n'est jamais versionné (modèle dans env/prod.env.example) ;
- Un build release sans --flavor prod (ou avec les défauts dev embarqués) lève une erreur au démarrage (fail-fast S2).

Définir le super admin

Le super admin est identifié par un rôle dans la base — la table public.super_admins dans Supabase (plus d'e-mail codé en dur). Pour promouvoir un utilisateur (SQL Editor du projet) :

```
INSERT INTO public.super_admins (user_id)
SELECT id FROM auth.users WHERE email = 'admin@yourdomain.com'
ON CONFLICT (user_id) DO NOTHING;
```

> L'app résout le rôle via la RPC is_super_admin ; le RLS côté serveur est l'autorité. Détails dans Multi-tenant et Super Admin.

Persistance active (Supabase × local)

Avec Supabase configuré (env/dev.env — projet dev), les modules métier sont écrits à distance par tenant et par magasin (RLS + RPC atomiques — datasources dans lib/data/datasources/remote/). Pour développer avec la persistance locale (SQLite sur natif / en mémoire sur web), exécutez avec des variables vides :

```
flutter run --dart-define=SUPABASE_URL= --dart-define=SUPABASE_ANON_KEY=
```

Signature de la release Android

La signature de release lit android/key.properties (jamais versionné — voir .gitignore et le modèle documenté android/key.properties.example).

- Keystore : android/app/upload-keystore.jks (déjà généré — alias upload, RSA 2048, valide jusqu'en 2053) ;
- `key.properties` : déjà rempli avec les identifiants générés (les mots de passe vivent dans le fichier — sauvegardez-le dans un gestionnaire de mots de passe avec le .jks ; perdre les deux signifie que vous ne pouvez pas publier de mises à jour) ;
- PKCS12 (JDK 9+) : le store et la clé partagent UN SEUL mot de passe — storePassword et keyPassword doivent être égaux ;
- Sans `key.properties` (CI/dev) : le build de release retombe sur le keystore debug — non publiable. Garde Gradle : un build prodRelease échoue à la configuration sans key.properties (un AAB de production ne peut jamais être signé debug) ;
- Play Console : publiez le premier AAB avec cette clé, activez *Play App Signing*, et enregistrez l'empreinte SHA-256 de la clé d'upload dans *Setup → App integrity* ;
- CI/CD : .github/workflows/ci.yml ne construit que le flavor dev (validation). Les artefacts publiables viennent de .github/workflows/release.yml (tag v* → Fastlane) : il décode le keystore depuis le secret KEYSTORE_BASE64, construit l'AAB prod (--obfuscate --split-debug-info) et l'upload vers Internal testing (compte de service depuis PLAY_SERVICE_ACCOUNT_JSON). Voir docs/pt/13-build-e-publicacao.md pour la liste complète des secrets.

Pour inspecter la clé :

```
keytool -list -v -keystore android/app/upload-keystore.jks -storepass YOUR_PASSWORD
```

Carte mentale du flux de développement

```
Nouvelle fonctionnalité :
  1. Entité dans le domaine (freezed)      → lib/domain/entities/
```

```
2. Contrat de repository              → lib/domain/repositories/
3. Cas d'usage + exceptions du domaine  → lib/domain/usecases/
4. Implémentation dans data           → lib/data/datasources/ (en mémoire + SQLite)
                                        lib/data/datasources/remote/ (Supabase)
                                        lib/data/repositories/
5. Enregistrer dans le ServiceLocator  → lib/core/di/service_locator.dart
6. Provider Riverpod                  → lib/presentation/controllers/
7. Écran/widget                       → lib/presentation/screens/ | widgets/
8. Tests (domaine → données → UI)     → test/
9. `dart run build_runner build --delete-conflicting-outputs`
```

Conventions du projet

Architecture (non négociable)

- Règle de dépendance : domain n'importe pas data, presentation ni l'infrastructure core ; les dépendances pointent toujours vers l'intérieur ;
- Contrats comme interfaces : les repositories et datasources du domaine sont des abstract interface class ; les implémentations vivent dans data ;
- UI mince : les règles métier dans les cas d'usage ; les écrans n'attrapent que les exceptions du domaine et affichent des messages ;
- GetIt uniquement à la composition root : c'est le ServiceLocator qui fournit les instances ; les providers font le pont vers Riverpod.

Persistance

- Toujours deux implémentations : en créant un datasource, faites la version en mémoire (*_impl.dart) et la SQLite (*_sqlite_impl.dart) implémentant le même contrat ;
- SQL paramétré (instructions préparées) — jamais de concaténation de chaînes ;
- Sérialisation : enums par nom, DateTime en millis epoch, bool en 0/1 ;
- Transactions : plusieurs écritures liées dans IUnitOfWork / runInTransaction (réentrant) ;
- Migrations légères : ALTER TABLE idempotent dans AppDatabase.createSchema, protégé par PRAGMA table_info ;
- Gardez les signatures synchronisées entre app_database_io.dart ↔ app_database_stub.dart.

SQLite pour de nouvelles données

En ajoutant des tables/colonnes : créez-les dans createSchema (avec CHECK de garde) et ajoutez une migration idempotente si nécessaire — rappelez-vous que la base démarre vide (pas de seed).

État (Riverpod)

- État asynchrone → AsyncNotifier/AsyncNotifierProvider (AsyncValue : loading/error/data) ;
- Providers « pont » pour les cas d'usage via ServiceLocator.instance<T>() ;
- refresh() = invalidateSelf() + await future ;

- Rechargez les listes dépendantes avec ref.invalidate(...) après les mutations (ex. : vente → produits, ventes, transactions et caisse).

UI

- Material 3, thème via AppTheme (AppColors pour les sémantiques : entrées/sorties/avertissement) ;
- Formatage pt-BR via Formatters (devise, date, jour relatif) ;
- Bottom sheets de formulaire via showAppSheet + AppSheetHeader ;
- États d'écran : EmptyState, ErrorView, KpiCard, StatusChip, SearchField, SectionHeader.

Tests

- Fakes de contrats dans les cas d'usage ; openTestDatabase() (SQLite en mémoire) pour les données ;
- Overrides de providers dans les tests d'écrans (ProviderScope(overrides: [...])) ;
- Chaque exception du domaine doit avoir un test.

Ajouter un nouveau module (étape par étape)

Exemple fictif : module Fournisseurs.

1. Domaine :
   2. Supplier (entité freezed) dans lib/domain/entities/ ;
   3. ISupplierRepository dans lib/domain/repositories/ ;
   4. GetSuppliersUseCase, AddSupplierUseCase dans lib/domain/usecases/ ;
5. Données :
   6. SupplierLocalDataSource (contrat) + SupplierLocalDataSourceImpl (en mémoire) + SupplierSqliteDataSourceImpl ;
   7. SupplierRepositoryImpl ;
   8. table suppliers dans AppDatabase.createSchema (+ migration s'il y a une ancienne base) ;
9. DI : enregistrez le datasource (dans les deux branches du ServiceLocator), le repository et les cas d'usage ;
10. Présentation : suppliers_providers.dart, écran, sheet d'inscription ;
11. Tests : cas d'usage, datasource SQLite, écran ;
12. dart run build_runner build --delete-conflicting-outputs et flutter test.

Commandes utiles

Commande	Objectif
flutter pub get	Installer les dépendances
dart run build_runner build --delete-conflicting-outputs	Générer le code Freezed/JSON

flutter analyze	Analyse statique (lints)
flutter test	Exécuter tous les tests
flutter test test/<file>	Exécuter un fichier de test
flutter run -d chrome --flavor dev --dart-define-from-file=env/dev.env	Exécuter sur le web
flutter build appbundle --flavor prod --release --dart-define-from-file=env/prod.env	AAB de production (signé)
bash scripts/sql_check.sh [PGHOST] [PGUSER] [PGDATABASE]	Harnais SQL/RLS/RPC (Postgres jetable)

Documents connexes

- Architecture — règles structurelles.
- Couche de données — schéma et conventions de persistance.
- Multi-tenant et Super Admin — forfaits, approbation des tenants et rôles.
- Tests — comment tester chaque couche.